

# Algoritmi e strutture dati

Alessandro Carella

# Indice

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>1</b> | <b>Algoritmi e programmi</b>                              | <b>3</b>  |
| 1.1      | Ciclo di sviluppo del software . . . . .                  | 3         |
| 1.1.1    | Qualità esterne . . . . .                                 | 4         |
| 1.1.2    | Qualità interne . . . . .                                 | 4         |
| 1.2      | Astrazione nei sistemi software . . . . .                 | 5         |
| 1.3      | Tecniche di progettazione . . . . .                       | 6         |
| 1.4      | Classificazione dei linguaggi di programmazione . . . . . | 7         |
| <b>2</b> | <b>Algebre di dati</b>                                    | <b>8</b>  |
| 2.1      | Astrazione funzionale . . . . .                           | 8         |
| 2.2      | Astrazione di dati . . . . .                              | 10        |
| 2.2.1    | Requisiti dell'astrazione di dati . . . . .               | 14        |
| 2.2.2    | Specificazione e realizzazione . . . . .                  | 14        |
| 2.3      | Struttura di dati . . . . .                               | 16        |
| 2.3.1    | Tecniche di specifica . . . . .                           | 18        |
| <b>3</b> | <b>Introduzione alla programmazione object oriented</b>   | <b>22</b> |
| 3.1      | Oggetti e classi . . . . .                                | 22        |
| 3.2      | Linguaggi Object Oriented . . . . .                       | 23        |
| 3.2.1    | Ereditarietà . . . . .                                    | 23        |
| <b>4</b> | <b>Liste</b>                                              | <b>25</b> |
| 4.1      | Strutture lineari . . . . .                               | 25        |
| 4.1.1    | Modalità di accesso . . . . .                             | 25        |
| 4.1.2    | Modalità di agire . . . . .                               | 26        |
| 4.2      | Lista . . . . .                                           | 26        |
| 4.2.1    | Specificazione sintattica . . . . .                       | 27        |
| 4.2.2    | Specificazione semantica . . . . .                        | 27        |

|       |                                         |    |
|-------|-----------------------------------------|----|
| 4.2.3 | Realizzazione . . . . .                 | 30 |
|       | Raprpresentazione sequenziale . . . . . | 30 |
|       | Rappresentazione collegata . . . . .    | 30 |

# Capitolo 1

## Algoritmi e programmi

### 1.1 Ciclo di sviluppo del software

Il ciclo di sviluppo del software, generalmente, è composto dalle seguenti attività.

- **Studio di fattibilità.** Questa fase ha lo scopo di valutare costi e benefici del sistema da costruire, quindi, vengono analizzati tempi e modi di attuazione, vengono valutate alternative possibili per lo sviluppo del sistema e delle risorse necessarie.
- **Raccolta e analisi dei requisiti.** In questa fase viene definito il problema e si specifica l'ambiente di sviluppo sia hardware che software. Alla fine viene prodotto un *documento di analisi* dove viene specificato l'hardware e il software scelto, la struttura concettuale dei dati da elaborare e le caratteristiche generali delle operazioni da fare.
- **Progettazione.** Nella fase di progettazione viene individuata la soluzione informatica del problema, si definisce l'architettura del programma specificandone la struttura, le funzioni delle componenti individuate. Inoltre, si sceglie le strutture di rappresentazione degli oggetti e si scrive il programma secondo il linguaggio e l'ambiente operativo scelto ottenendo il **codice**.
- **Verifica.** In questa fase si analizza sia la documentazione prodotta che il programma realizzato, vengono, quindi, eseguite prove di correttezza sintattiche e logiche.

- **Manutenzione.** Durante il periodo di esercizio, si verifica che il programma produca i risultati attesi e se necessario, aggiornare il programma.

La fase di progettazione deve fornire un'analisi delle qualità che il programma deve possedere:

- **Esterne.** Si riferiscono a caratteristiche evidenziabili dal solo funzionamento del programma durante la fase di esercizio.
- **Interne.** Si riferiscono alle caratteristiche analizzabili e valutabili da esperti, attraverso uno studio delle scelte tecniche adottate.

### 1.1.1 Qualità esterne

Le qualità esterne del programma sono:

- **Correttezza.** La correttezza è la capacità di eseguire precisamente i compiti individuati durante l'analisi dei requisiti.
- **Efficienza.** L'efficienza è la capacità di usare in modo razionale ed economico le risorse di calcolo.
- **Robustezza.** La robustezza è la capacità di funzionare in modo soddisfacente in condizioni limite o anomale rispetto a quelle previste in fase di analisi dei requisiti.
- **Usabilità.** L'usabilità è la capacità di consentire un'interazione semplice, naturale ed efficace con l'utente finale.

### 1.1.2 Qualità interne

Le qualità interne del programma sono:

- **Riusabilità.** La riusabilità è la capacità di essere riutilizzato, in tutto o in parte, per applicazioni diverse rispetto a quella per la quale è stato prodotto.
- **Modularità.** La modularità è il grado di organizzazione interna del programma, quindi la strutturazione delle singole parti, della funzionalità e del modo in cui cooperano per l'obiettivo generale.
- **Estensibilità.** L'estensibilità è la capacità di adattarsi facilmente a modifiche nei requisiti.

- **Portabilità e compatibilità.** Essa è la facilità di trasferire il software prodotto in ambiti diversi.
- **Leggibilità.** Essa è la capacità del codice di essere autoesplicante.
- **Bontà della documentazione.** Essa è la completezza ed efficacia dei documenti annessi.

## 1.2 Astrazione nei sistemi software

Una descrizione specifica di un sistema pone enfasi su alcuni dettagli o proprietà e ne elimina altri. Il processo di astrazione prevede il decidere:

- quali caratteristiche sono rilevanti;
- quali parametri andrebbero inclusi;
- quale formalismo descrittivo andrebbe adottato;
- come validare il modello.

Come in altri campi, ci sono gerarchie di modelli e i livelli più bassi forniscono descrizioni più esaurienti rispetto ai livelli superiori.

In programmazione ci si riferisce alla descrizione astratta fornita da un modello come alla specifica e al livello più bassi nella gerarchia di modelli come alla realizzazione.

Le astrazioni usate per il software tendono ad evidenziare gli aspetti funzionali, quindi cosa è calcolato piuttosto che come è condotto il procedimento di calcolo. L'astrazione fornisce una distinzione tra il livello di *concettualizzazione* e quello di *realizzazione*. Nella fase di *concettualizzazione* viene individuata la soluzione al problema, vengono specificati gli oggetti da trattare e vengono ideate le scelte algoritmiche.

La fase di *realizzazione* si riferisce a quel processo che, partendo dallo schema concettuale, permette di ottenere uno specifico programma che realizza gli elementi dello schema e questo impone la scelta dei moduli software e della loro realizzazione.

L'astrazione viene usata per decomporre, rappresentare e generalizzare. La scomposizione di un problema in sottoproblemi è un metodo semplice per affrontare problemi complessi e l'astrazione è usata per cogliere gli aspetti essenziali di

una situazione del mondo reale in modo da definire le opportune strutture di rappresentazione che ne evidenziano alcuni aspetti e ne trascurino altri.

Ci sono due tipi di astrazione:

- **Astrazione sui dati.** Consente di far riferimento a strutture algebrico-matematiche, caratterizzate da valori e da operazioni su tali valori. Si prescinde dal modo in cui queste strutture sono organizzate e realizzata mediante i costrutti di un linguaggio di programmazione.
- **Astrazione sulle funzioni.** Consente di concentrare l'attenzione su cosa fa una particolare operazione piuttosto che sul come è fatta. Si astrae dalle modalità di realizzazione, ci si concentra sul compito o sulle funzionalità di un segmento di programma.

sui dati e sulle funzioni.

### 1.3 Tecniche di progettazione

Le tecniche di specifiche consentono di esprimere gli elementi essenziali dello schema concettuale mediante formalismi grafici o un linguaggio basato sulla logica e sull'algebra per descrivere gli aspetti concettuali dei tipi astratti di dato. Le tecniche di programmazione riguardano i metodi per la strutturazione e per la stesura dei programmi a partire dallo schema concettuale e possono essere:

- programmazione strutturata, con schemi iterativi e ricorsivi;
- programmazione orientata agli oggetti;
- programmazione logica e funzionale.

La *modularizzazione* consente di razionalizzare lo sviluppo del software costruendo programmi costituiti da parti indipendenti e interagenti. Un modulo è caratterizzato da una struttura interna, è definito con un determinato scopo, offre all'esterno un insieme di funzionalità utilizzabili da altri moduli e ha precise relazioni con altri moduli. La modularizzazione facilita il processo di sviluppo e le attività di analisi e di verifica e favorisce la riusabilità. La qualità della modularizzazione aumenta:

- all'aumentare della **coesione** di un modulo in quanto incapsula un insieme di caratteristiche omogenee, indipendenti da altri moduli;

- all'aumentare dell'uso di **information hiding**, i dettagli interni al modulo non devono giocare alcun ruolo nell'utilizzo del modulo da parte di moduli esterni;
- al diminuire dell'**accoppiamento** tra moduli. Non si creano dipendenze non volute o non necessarie.
- con l'utilizzo dell'**interfacciamento esplicito** che suggerisce di rappresentare mediante parametri tutti i dati che vengono scambiati tra due sottoprogrammi.

Le *tecniche di progettazione di algoritmi e strutture dati* riguardano la definizione degli algoritmi e la individuazione delle strutture dati piu' appropriate per un determinato problema, come tecniche enumerative, backtracking, divide et impera, ecc.

## 1.4 Classificazione dei linguaggi di programmazione

- **Linguaggi imperativi.** Il programma è un insieme di istruzioni che corrispondono a precisi comando impartiti ad una macchina che li esegue sequenzialmente.
- **Linguaggi di programmazione funzionale.** Il programma è la specifica di una funzione che, in base a un insieme di dati di ingresso, calcola il risultato secondo una legge specificabile in modo matematico.
- **Linguaggi di programmazione logica.** Il programma è la specifica di una relazione che sussiste tra un insieme di dati e la specifica è costruita mediante un sistema formale basato sulla logica.
- **Linguaggi di programmazione O.O.** Il programma è la specifica di un insieme di oggetti che rappresentano gli elementi della situazione in gioco in un certo problema, e ciascun oggetto è specificato in termini di una struttura e di un insieme di operazioni tramite le quali si ottiene il comportamento voluto per risolvere il problema.

# Capitolo 2

## Algebre di dati

### 2.1 Astrazione funzionale

Un programma corrisponde alla tripla  $\{D, A, R\}$  ove:

- $D$  corrisponde ai dati in input al programma;
- $A$  corrisponde all'implementazione del programma;
- $R$  corrisponde ai risultati ottenuti dal programma.

Si può dire che, poichè trasforma i dati iniziali in risultati, definisce un nuovo *operatore* sui dati e il repertorio di operatori disponibili sui dati può essere ampliato scrivendo programmi.

L'*astrazione funzionale* è la tecnica che permette di potenziare il linguaggio disponibile introducendo nuovi operatori:

- questi sono definiti attraverso sotto-programmi in cui si fa degli operatori di base già disponibili;
- l'intestazione del sotto-programma introduce il nome e gli argomenti dell'operatore, mentre il corpo ne descrive le regole di comportamento.

#### **Esempio : Funzione per il calcolo del coefficiente binomiale**

Consideriamo la formula per il calcolo del coefficiente binomiale:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

che fornisce il numero di possibili combinazioni di  $n$  oggetti presi a gruppi di  $k$ .

Il calcolo è immediato se si suppone di avere un operatore  $fatt(i)$  che calcola il fattoriale di un valore  $i$ .

```
1 int binomial(int n, int k)
2   return fatt(n) / fatt(k) * fatt(n-k);
```

Se  $fatt(i)$  non è disponibile è necessario scrivere un sotto-programma per ampliare il repertorio degli operatori e l'astrazione funzionale ci consente questo.

I linguaggi di programmazione ad alto livello permettono l'uso di astrazione funzionale, cioè consentono di:

- creare delle unità di programma dando un nome ad un gruppo di istruzioni e stabiliscono le modalità di comunicazione tra l'unità di programma creata ed il resto del programma in cui essa si inserisce;
- al momento della attivazione (su chiamata) della unità del programma viene sospesa l'esecuzione del programma chiamante: il controllo passa all'unità attivata e, all'atto del completamento della sua esecuzione, l'attivazione termina e il controllo torna al programma chiamante.

I costrutti linguistici per realizzare l'astrazione funzionale consentono di definire una specifica e una realizzazione:

- la specifica definisce "cosa" ci si aspetta dalla funzione, cioè definirà la relazione che caratterizza il legame tra dati di ingresso e risultati. Nel caso del fattoriale la specifica dirà che l'intestazione è "int fatt(int k)"; l'argomento  $k$  è una variabile di tipo intero e il risultato è ancora di tipo intero. La funzione da calcolare è definita come:  $fatt(k) = k(k-1) \cdots 2 \cdot 1$
- La realizzazione definisce come il risultato viene ottenuto. Alla stessa specifica potranno corrispondere scelte di realizzazione diverse. La funzione "fatt" potrebbe essere realizzata con un programma iterativo:

```
1 fatt(int k)
2   f = 1
3   for i = 2 to k do
4     f = f * i
5   return f
```

ma anche come un programma ricorsivo:

```
1 fatt(int k)
2   if k = 1 then f = 1
3   else f = k * fatt(k - 1)
4   return f
```

## 2.2 Astrazione di dati

La astrazione di dati ricalca ed estende il concetto di astrazione funzionale. Così come l'astrazione funzionale permette di ampliare l'insieme di modi di operare sui dati, la astrazione di dati permette di ampliare i tipi di dati disponibili attraverso l'introduzione sia di nuovi tipi di dati che di nuovi operatori.

L'astrazione funzioanle stimola gli sforzi per evidenziare operazioni ricorrenti o ben caratterizzate all'interno della soluzione di un problema.

L'astrazione di dati sollecita ad individuare le organizzazioni dei dati piu' adatte alla soluzione del problema.

Un'astrazione di dati consente una estensione dell'algebra dei dati disponibile in un linguaggio di programmazione. Un'algebra è un sistema matematico costituito da un dominio e da un insieme di funzioni applicabili su tali valori: il numero, il tipo e le proprietà delle funzioni sono gli elementi essenziali di un algebra.

La corrispondenza tra tipo astratto e algebra si basa sul parallelo tra gli insiemi di valori di un algebra e i domini di definizione di un tipo astratto, e tra le funzioni di un'algebra e le operazioni associate a un tipo astratto.

Molti linguaggi di programmazione forniscono i mezzi per definire e creare nuovi tipi di dati, ma non tutti consentono di fare astrazione dati o creare tipi di dati astratti. Un tipo di dati si dice astratto se le operazioni applicabili sugli oggetti che li rappresentano sono isolate dai dettagli usati per realizzare il tipo.

Un'algebra di dati puo' essere definita come costituita da:

- A) una famiglia di insiemi, cioè l'insieme dei dati;
- B) una famiglia di operatori sui dati;
- C) un repertorio di simboli o nomi per indicare l'insieme dei dati;
- D) un repertorio di simboli o nomi per indicare gli operatori;
- E) un repertorio di simboli, detti *costanti*, per indicare elementi singoli degli insiemi dei dati.

### Esempio : Esempio di un'algebra di dati

- A) La famiglia di **insiemi** comprende: i numeri interi, i booleani, le stringhe;
- B) la famiglia di **operatori** comprende:
  - operatori aritmetici come somma, sottrazione, moltiplicazione e divisione;
  - operatori di congiunzione, disgiunzione e negazione per i booleani;
  - operatori di confronto tra interi come maggiore, minore, uguale ecc.;
  - operatori di concatenazione per le sequenze di caratteri e di selezione di sottostringa
- C) il repertorio dei simboli o nomi per l'insieme dei dati potrebbe essere: integer, boolean, string;
- D) il repertorio dei simboli o nomi per gli operatori potrebbe essere:
  - +, -, \*, / per gli operatori aritmetici;
  - and, or, not per gli operatori booleani;
  - <, >, = per i confronti tra interi;
  - substr e & per la selezione e concatenazione di stringhe.
- E) le costanti per gli insiemi di dati potrebbero essere:
  - ogni intero viene indicato da una sequenza di cifre decimali, eventualmente precedata da + o -, interpretata secondo la notazione decimale;
  - ogni sequenza di caratteri viene indicata dalla sequenza racchiusa tra apici;
  - i valori booleani sono indicati da true e false.

Gli operatori possono essere combinati per ottenere delle espressioni. Le caratteristiche di un'algebra dipendono dalla famiglia di insiemi e dalla famiglia di operatori.

Nei linguaggi di programmazione il binomio "dati-operatori" è inscindibile. Dun-

que, è necessario definire un tipo di dati come un insieme di dati piu' gli operatori che ad esso si riferiscono, nel contesto di una definita algebra.

Anche un'astrazione di dati è costituita da:

- una **specificazione**: ha il compito di descrivere sintenticamente il tipo dei dati e gli operatori che li caratterizzano;
- una **realizzazione**: stabilisce come i dati e gli operatori vengono ricondotti ai tipi di dati e agli operatori già disponibili.

La scelta delle strutture dati è il primo passo per un buon risultato nell'attività di programmazione.

### Esempio : Un esempio di astrazione dati

Supponiamo di voler effettuare un'astrazione di dati aggiungendo ai tipi di dati standard i numeri complessi.

I numeri complessi hanno forma:  $r_1 + i \cdot r_2$  ove  $r_1$  e  $r_2$  sono numeri reali e  $i = \sqrt{-1}$ .

Per denotarli matematicamente basta la coppia  $(r_1, r_2)$  dove  $r_1$  è il coefficiente della parte reale e  $r_2$  quello della parte immaginaria.

è necessario definire operatori di selezione e costruzione che operano su complessi e su reali, operatori di somma e prodotto che operano esclusivamente sui complessi.

Il calcolo di somma e prodotto su numeri complessi non è difficile se ricordiamo che:

- la somma di due numeri complessi  $c_1$  e  $c_2$  dà luogo ad un nuovo numero complesso  $c_3$  la cui parte reale è la somma delle parti reali e la cui parte immaginaria è somma delle parti immaginarie:
  - $\text{parteReale}(c_3) = \text{parteReale}(c_1) + \text{parteReale}(c_2)$
  - $\text{parteImmg}(c_3) = \text{parteImmg}(c_1) + \text{parteImmg}(c_2)$
- il prodotto di due numeri complessi  $c_1$  e  $c_2$  dà luogo ad un nuovo numero complesso  $c_3$ , la cui parte reale è la differenza tra il prodotto della parte reale del primo numero per la parte reale del secondo numero e il prodotto della parte immaginaria del primo numero per la parte immaginaria del secondo numero:
  - $\text{parteReale}(c_3) = \text{parteReale}(c_1) * \text{parteReale}(c_2) - \text{parteImmg}(c_1) * \text{parteImmg}(c_2)$

e la cui parte immaginaria è somma del prodotto della parte reale del primo numero per la parte immaginaria del secondo numero con il prodotto della parte immaginaria del primo numero con la parte reale del secondo numero:

$$\text{parteImmg}(c_3) = \text{parteReale}(c_1) * \text{parteImmg}(c_2) + \text{parteReale}(c_2) * \text{parteImmg}(c_1)$$

Passando agli operatori necessari, abbiamo bisogno di definire, attraverso procedure e funzioni:

- $\text{cxsum}(c_1, c_2) = c_3$  cioè la somma di complessi;
- $\text{cxprod}(c_1, c_2) = c_3$  cioè il prodotto di complessi;
- $\text{realpart}(c) = r$  che seleziona la parte reale del complesso  $c$ ;
- $\text{imgpart}(c) = i$  che seleziona la parte immaginaria del complesso  $c$ ;
- $\text{conscx}(r_1, r_2) = c$  che è la combinazione di due reali, il primo come parte reale, il secondo come immaginaria del complesso  $c$ .

Per poter fare astrazione dati, bisogna disporre di un linguaggio di programmazione a tipizzazione forte come il pascal o il C.

In Pascal, infatti, si può dichiarare:

```
1 TYPE
2   COMPLEX = ARRAY[2] OF REAL;
3   VAR
4     C1, C2, C3, C4: COMPLEX;
5   PROCEDURE SOMMACOMPLESSI(
6     C1, C2: COMPLEX;
7     VAR C3: COMPLEX);
```

oppure:

```
1 TYPE
2   COMPLEX = RECORD OF
3     PARTEREALE: REAL;
4     PARTEIMMAGINARIA: REAL
5   END;
```

Disponendo di un linguaggio con tipi, possiamo utilizzare dati che definiamo e dichiariamo direttamente come complessi, prescindendo dalla effettiva realizza-

zione, e fare in modo che alle procedure costruite per i nostri dati abbiano accesso esclusivamente a quei dati.

Queste caratteristiche necessarie a una buona astrazione di dati sono note come *i requisiti dell'astrazione di dati*.

### 2.2.1 Requisiti dell'astrazione di dati

- Il **requisito di astrazione** è verificato quando i programmi che usano un'astrazione possono essere scritti in modo da non dipendere dalle scelte di realizzazione.

La mancanza del requisito di astrazione si manifesta con l'impossibilità di dichiarare direttamente variabili con il nuovo tipo definito, quando non posso fare riferimento ai nuovi dati creati prescindendo dalla rappresentazione usata e dalla realizzazione;

- Il **requisito di protezione** è verificato se sui nuovi dati si può lavorare esclusivamente con gli operatori definiti all'atto della specifica.

La mancanza del requisito di protezione si manifesta con la possibilità di lavorare con gli operatori definiti per i nuovi dati anche su dati con rappresentazioni simili, ma non omogenei per tipo.

Il requisito di astrazione, in accordo con i principi dell'astrazione di dati, impone che i programmi siano sensibili solo a variazioni della specifica dei nuovi tipi. Il requisito di protezione è particolarmente importante, se vogliamo che l'esecutore del linguaggio assicuri i controlli di consistenza tra i tipi di dati e gli operatori, non solo ai dati primitivi, ma anche ai dati ottenuti per astrazione.

### 2.2.2 Specifica e realizzazione

Si è detto che un'astrazione di dati è costituita da una **specifica** e una **realizzazione**.

Una specifica, che ha il compito di descrivere sinteticamente il tipo dei dati e gli operatori che li caratterizzano, si distingue in:

- **specifica sintattica** che fornisce:
  - l'elenco dei nomi dei tipi di dato utilizzati per definire la struttura, delle operazioni specifiche della struttura e delle costanti;

- i domini di partenza e di arrivo, cioè i tipi degli operandi e del risultato per ogni nome di operatore.
- **specificazione semantica** che definisce il significato dei nomi introdotti con la specifica sintattica. Associa:
  - un insieme ad ogni nome di tipo introdotto nella specifica sintattica;
  - un valore ad ogni costante;
  - una funzione ad ogni nome di operatore esplicitando le seguenti condizioni sui domini di partenza e di arrivo:
    - \* **precondizione** che definisce quando l'operatore è applicabile;
    - \* **postcondizione** che stabilisce come il risultato sia vincolato agli argomenti dell'operatore.

La realizzazione sfrutta al meglio le possibilità offerte dall'ambiente di programmazione. La definizione di un formalismo per le specifiche sintattiche non pone particolari problemi. L'individuazione di formalismi per le specifiche semantiche è più complicata, in genere le specifiche semantiche sono date con frasi estratte dal linguaggio naturale o da quello matematico. Vedere dalle slide l'esempio di specifica.

La realizzazione riconduce la specifica ai tipi di dato primitivi e agli operatori già disponibili, in particolare gli operatori sono simulati con sottoprogrammi. Alla stessa specifica possono corrispondere diverse realizzazioni più o meno efficienti. Nel valutare l'efficienza di sottoprogrammi che utilizzano tipi di dato primitivi, si procede dalle caratteristiche della macchina, assumendo di far riferimento ad una organizzazione molto semplice. Si fa riferimento a:

- dati contenuti in memoria;
- memoria organizzata in celle (parola);
- celle accessibili tramite indirizzo;
- indirizzi che sono interi consecutivi;
- dati elementari che non decidono la capacità di una cella;
- variabili contenute in celle di indirizzo noto il cui accesso richiede tempo costante.

Un costrutto di programmazione propriamente adatto alla realizzazione dati dovrà:

- dichiarare esplicitamente quali sono i nuovi tipi di dati e quali sono i nuovi operatori;
- definire la rappresentazione dei nuovi dati in termini di dati pre-esistenti;
- contenere un insieme di sottoprogrammi che realizzino gli operatori definiti sui nuovi dati.

## 2.3 Struttura di dati

Il tipo di dato è un modello matematico che consiste nella definizione di una collezione di valori sui quali sono ammesse certe operazioni.

Una struttura di dati è un particolare tipo di dato, caratterizzato più dalla organizzazione imposta agli elementi che la compongono che dal tipo degli elementi componenti.

Le strutture solitamente disponibili nei linguaggi di programmazione sono le tabelle monodimensionali, come vettori o array, i cui elementi sono contenuti in memoria centrale in locazioni contigue. In generale, distinguiamo strutture:

- **lineari**: dati disposti in sequenza;
- **non lineari**: in cui non è individuata una sequenza;
- **a dimensione fissa (statiche)**: in cui il numero di elementi non può variare nel tempo;
- **a dimensione variabile (dinamiche)**: in cui il numero di elementi può variare nel tempo;
- **omogenee**: dati dello stesso tipo;
- **non omogenee**: dati non dello stesso tipo.

### Esempio : Esempio di struttura dati

La classica struttura vettore (array) è a dimensione fissa. è una tabella monodimensionale di elementi omogenei, su cui possono effettuarsi operazioni di:

- lettura o selezione: reperimento del valore di un elemento;

- scrittura o sostituzione di un valore di un elemento con un nuovo valore.

Un vettore di  $n$  componenti è inteso come sequenza di oggetti in relazione d'ordine tra loro e l'organizzazione in memoria consente l'accesso diretto al singolo componente attraverso l'indice.

Se volessimo definire le specifiche di un vettore:

- Specifica sintattica:

```

1 Tipi:
2   vettore, intero, tipoelem
3 Operatori:
4   creavettore: () -> vettore
5   leggivettore: (vettore, intero) -> tipoelem
6   scrivivettore: (vettore, intero, tipoelem) -> vettore

```

- Specifica semantica:

```

1 Tipi:
2   intero: l'insieme dei numeri interi
3   vettore: l'insieme delle sequenze di  $n$  elementi di tipo
4   tipoelem
5 Operatori:
6   creavettore =  $v$ 
7     post: per ogni  $i$ ,  $1 \leq i \leq n$ , l' $i$ -esimo elemento
8     del vettore  $v(i)$  è uguale ad un definito elemento
9     di tipo tipoelem
10  leggivettore( $v$ ,  $i$ ) =  $e$ 
11    pre:  $1 \leq i \leq n$ 
12    post:  $e = v(i)$ 
13  scrivivettore( $v$ ,  $i$ ,  $e$ ) =  $v'$ 
14    pre:  $1 \leq i \leq n$ 
15    post:  $v'(i) = e$ ,  $v'(j) = v(j)$ 
16    per ogni  $j$  tale che  $1 \leq j \leq n$  e  $j \neq i$ 

```

- Realizzazione:

In molti linguaggi il vettore è il tipo di dato primitivo:

- in Fortran creavettore corrisponde a dimension  $v(n)$ , leggivettore( $v$ ,  $i$ ) corrisponde a  $v(i)$  e scrivivettore( $v$ ,  $i$ ,  $e$ ) corrisponde a  $v(i) = e$
- in Pascal corrisponde all'array, creavettore corrisponde a var  $v$ : array[1.. $n$ ] of type tipoelem, leggivettore( $v$ ,  $i$ ) corrisponde a  $v[i]$  e scrivivettore( $v$ ,  $i$ ,  $e$ ) corrisponde a  $v[i] := e$

Vedere esempio della matrice dalle slide.

### 2.3.1 Tecniche di specifica

Due tecniche comuni per la scrittura di specifiche sono:

- specifiche assiomatiche o algebriche;
- modelli astratti (approccio costruttivo).

In entrambe le tecniche la sintassi di ogni operatore viene specificata in termini del nome dell'operatore e dei tipi di parametri. La differenza fra le due sta nel come viene definita la semantica degli operatori.

Le specifiche assiomatiche sono self-contained e specificano ogni oggetto come composizione di funzioni.

I modelli astratti definiscono la semantica delle operazioni in termini di un altro ben definito tipo di dato. La chiave di queste e di altre tecniche di specifica sta nel fatto che la descrizione della semantica non fa riferimento alla realizzazione. Possiamo definire formalmente una specifica di un tipo di dato astratto come una tripla  $(D, F, A)$ :

- $D$ : l'insieme di tutti i domini per la definizione del dato.  
Il tipo di dato che stiamo definendo, indicato con il simbolo  $d$ , è chiamato **dominio designato**; tutti gli altri domini che fanno parte del dato sono detti **domini ausiliari**.  
Nel caso dello Stack,  $D$  potrebbe essere Stack, ItemType, Boolean ove  $d = \text{Stack}$  e i domini ausiliari sono ItemType, Boolean
- $F$  è l'insieme degli operatori;
- $A$  è l'insieme degli assiomi o regole che descrivono la semantica degli operatori, nel caso dello Stack dovrebbero definire la proprietà LIFO dello stack.

#### Tipi di operatori

Gli operatori si suddividono in:

- **Costruttori** (Constructors): costruiscono/inizializzano una nuova istanza del dato;

- **Modificatori** (Transformers/Modifiers): operatori che cambiano la struttura in qualche modo;
- **Osservatori** (Observers/Accessors): permettono di osservare lo stato del dato senza modificarlo
- **Distruttori** (Destructors): puliscono l'occupazione di memoria del dato e la rilasciano per il riuso;
- **Iteratori** (Iterators): sono usati per permettere all'utente di processare l'intera struttura, componente per componente.

### Esempio : Esempio di specifica assiomatica

Le specifiche assiomatiche definiscono il comportamento di un tipo di dato astratto dando gli **assiomi** che legano tra loro gli operatori.

Vediamo un esempio di una specifica algebrica per il tipo Stack:

|                    |                       |            |
|--------------------|-----------------------|------------|
| <b>struttura</b>   | Stack(di ItemType)    |            |
| <b>interfaccia</b> | Create                | → Stack    |
|                    | Push(Stack, ItemType) | → Stack    |
|                    | Pop(Stack)            | → Stack    |
|                    | Top(Stack)            | → ItemType |
|                    | IsEmpty(Stack)        | → Boolean  |

d = Stack(di ItemType)

D = Stack, ItemType, Boolean

F = Create, Push, Pop, Top, IsEmpty

L'interfaccia elenca le funzioni, il tipo dei parametri e tipi per i risultati. Non c'è nessuna indicazione su cosa faccia ogni funzione, la semantica degli operatori viene dato con gli assiomi.

Ogni istanza del dato astratto Stack puo' essere denotata come una sequenza di applicazioni di operatori detta **espressione**, ad esempio: Push(Push(Push(Create, 1), 2), 3)

Gli operatori observer non fanno parte di una espressione, in quanto vengono applicati ad una espressione perchè non modificano la struttura.

Gli assiomi costituiscono la terza componente della nostra tripla:

1. Gli assiomi per gli observer sono scritti in termini dei costruttori;
2. I costruttori che rimuovono un elemento dalla struttura sono definiti in termini dei costruttori che aggiungono un elemento alla struttura;
3. I costruttori che creano la struttura o aggiungono un elemento sono definiti esplicitamente.

Per la struttura Stack avremo, **per ogni S in Stack, i in ItemType**:

- $\text{IsEmpty}(\text{Create}) = \text{True}$
- $\text{IsEmpty}(\text{Push}(S, i)) = \text{False}$
- $\text{Top}(\text{Create}) = \text{Error}$
- $\text{Top}(\text{Push}(S, i)) = i$
- $\text{Pop}(\text{Create}) = \text{Error}$
- $\text{Pop}(\text{Push}(S, i)) = S$

**Esempio : Esempio di specifica costruttiva o modello astratto**

**Specifica sintattica:**

**struttura**      $\text{Stack}(\text{di ItemType})$

**interfaccia**      $\text{Create} \rightarrow \text{Stack}$   
                          $\text{Push}(\text{Stack}, \text{ItemType}) \rightarrow \text{Stack}$       $d = \text{Stack}(\text{di ItemType}), \text{l'in-}$   
                          $\text{Pop}(\text{Stack}) \rightarrow \text{Stack}$   
                          $\text{Top}(\text{Stack}) \rightarrow \text{ItemType}$   
                          $\text{IsEmpty}(\text{Stack}) \rightarrow \text{Boolean}$

sieme di tutte le istanze di Stack.

ItemType è l'insieme di tutti gli item che possono essere presenti in uno stack.

Boolean è l'insieme dei valori di verità true e false.

**Specifica semantica:**

In questo approccio diamo la semantica di ogni operatore associando ad ognuno una precondizione e una postcondizione. La precondizione è un'asserzione logica che specifica le caratteristiche richieste dei valori dei parametri, la postcondizione è un'asserzione logica che specifica le caratteristiche del risultato rispetto agli argomenti.

- $\text{Push}(S, I) \rightarrow S'$ :
  - PRE: -
  - POST:  $S'$  è lo stack S con I al top
- $\text{Pop}(S) \rightarrow S'$ :
  - PRE:  $\text{not empty}(S)$
  - POST:  $S'$  è lo stack S con l'elemento top eliminato

- $\text{Top}(S) \rightarrow I$ :
  - PRE:  $\text{not empty}(S)$
  - POST:  $I$  è l'elemento al top dello stack  $S$

## Capitolo 3

# Introduzione alla programmazione object oriented

Nella programmazione imperativa è possibile accedere alle variabili globali da qualsiasi parte del programma ma nei programmi molto grandi questa tecnica diventa ingestibile.

Pertanto è stato pensato di incapsulare ogni variabile globale in un modulo insieme ad un gruppo di operatori che possono accedere a tali variabili e gli altri moduli possono accedere indirettamente alle variabili solo per mezzo di questi operatori.

La programmazione Object-Oriented è una disciplina che si basa su oggetti per imporre una struttura modulare sui programmi.

### 3.1 Oggetti e classi

Un oggetto è definito come un gruppo di procedure che condividono uno stato. Quindi un oggetto è una collezione di dati che rappresentano gli *stati dell'oggetto* e procedure in grado di alterare lo stato, il *comportamento*.

Agli oggetti sono associate operazioni e valori.

#### Esempio : Coda

La Coda è la classe di tutti gli oggetti coda,  $q$  è un oggetto della classe Coda e sono definite le seguenti operazioni: creaCoda, aggiungi( $q$ ,  $i$ ), primo( $q$ ), rimuovi( $q$ ) e CodaVuota( $q$ ).

Nei linguaggi orientati agli oggetti, creaCoda è detto **costruttore** e crea

l'oggetto  $q$ . Anche il **distruttore**, che distrugge un oggetto, fa parte delle operazioni applicabili su un oggetto.

## 3.2 Linguaggi Object Oriented

Un linguaggio object-based supporta:

- **information hiding (encapsulation);**
- **Data abstraction;**
- **Message passing (polymorphism).**

Un linguaggio object-oriented implementa anche l'**ereditarietà**, incluso il dynamic binding, dove gli oggetti vengono organizzati secondo una gerarchia di classi e un oggetto può ereditare le proprietà della sua classe genitore.

Un oggetto è un'entità dinamica che potrebbe cambiare ma resta sempre lo stesso oggetto. Un programmatore object oriented affronta un problema dividendolo in oggetti che fanno qualcosa ed interagiscono fra loro.

Gli oggetti sono costituiti da dati privati e da operazioni permesse su tali dati e gli oggetti comunicano fra loro con il passaggio di messaggi che non sono altro che chiamate a delle procedure, dette *metodi*, che appartiene all'oggetto e potrebbe essere nascosta all'utente.

Una classe è una descrizione di oggetti da istanziare e nei linguaggi object oriented una classe è una collezione di oggetti, in cui ogni oggetto della classe include gli stessi metodi e le stesse variabili ma può includere diversi valori per i dati.

### 3.2.1 Ereditarietà

I linguaggi object-oriented, come detto precedentemente, supportano l'**ereditarietà** ovvero gli oggetti vengono organizzati secondo una gerarchia di classi e un oggetto può ereditare le proprietà della classe genitore. Gli attributi di una classe ereditata possono essere:

- **ridefiniti**: un attributo che ha lo stesso nome come in un superclasse, ma è definito in una sottoclasse;
- **specifici**: un attributo definito solo in una sottoclasse;
- **ereditati**: un oggetto ha un attributo definito solo in una delle sue superclassi.

Alcuni oggetti potrebbero ereditare piu' classi e un problema che si potrebbe verificare è il **name clashes** fra i metodi, ovvero se c'è un metodo con lo stesso nome nelle classi ereditate. In questo caso, in alcuni linguaggi object-oriented come il C++, ci sono operatori di scope resolution, :: in C++, per decidere quale metodo utilizzare.

# Capitolo 4

## Liste

### 4.1 Strutture lineari

Consideriamo strutture di dati che si sviluppano in una dimensione possono essere considerate come sequenze di oggetti, cioè *strutture lineari di dati*. In esse è importante l'esistenza di una relazione d'ordine tra gli oggetti che ci aiuta a individuarli e a selezionarli. Per distinguere le diverse strutture, bisogna considerare:

- le modalità di accesso ai dati;
- i modi di agire nella posizione individuata.

#### 4.1.1 Modalità di accesso

Le modalità di accesso ai dati all'interno di una struttura possono essere di diverso tipo.

- Accesso diretto.
- Accesso per scansione.
- Accesso agli estremi.

Il vettore, l'array, è tipico esempio di accesso diretto in quanto consente di accedere al singolo componentne mediante il nome e il meccanismo dell'indice.

La lista è un esempio di accesso per scansione in quanto consente l'accesso all'elemento generico solo dopo aver scandito gli elementi che lo precedono.

La pila e la coda sono esempi di accesso agli estremi dove la pila ha una filosofia LIFO e la coda ha una filosofia FIFO.

### 4.1.2 Modalità di agire

Relativamente ai modi di agire nelle posizioni individuate possiamo elencare i seguenti tipi di operazione.

- Lettura del valore di un componente, detto *ispezione*;
- Aggiornamento del valore di un componente, detto *cambio di valore*;
- Inserimento di un nuovo componente, detto *scrittura*;
- Rimozione di un componente, detto *cancellazione*.

Sono possibili diverse combinazioni tra i modi di accedere e i modi d'operare, l'unico problema riguarda la combinazione tra accesso diretto a tutta la struttura e possibilità di scrittura e cancellazione di componenti.

Generalmente queste operazioni alternano la numerazione dei componenti e questo è in contrasto col principio che i componenti siano individuati dal numero d'ordine della loro posizione.

Tutte le strutture dati lineari, escluso il vettore, sono dotate di operatori di inserimento e rimozione di componenti.

## 4.2 Lista

Una lista è una sequenza finita, anche vuota, di elementi dello stesso tipo in cui è possibile aggiungere o togliere elementi. Per fare questo, occorre specificare la posizione relativa all'interno della sequenza nella quale il nuovo elemento va aggiunto o dalla quale il vecchio elemento va tolto. La lista è una *struttura dati dinamica*, cioè a dimensione variabile e su può accedere direttamente solo ad un ristretto sottoinsieme di elementi, di solito il primo e per accedere ad un generico elemento occorre scandire sequenzialmente gli elementi della lista.

è bene notare che in una lista, a differenza di un insieme, uno stesso elemento può apparire più volte e gli elementi della lista, ai quali sono associati delle informazioni, sono definiti *atomi* o *nodi*.

Indichiamo la lista con la notazione:

$$l = \langle a_1, a_2, \dots, a_n \rangle \quad n \geq 0$$

La lunghezza di una lista è il numero dei suoi elementi. Se il numero di elementi è zero la lista si dice vuota. Inoltre, la lunghezza conta le posizioni, non i simboli distinti, così un simbolo che compare  $k$  volte contribuisce con  $k$  unità alla lunghezza della lista.

Se  $l = \langle a_1, a_2, \dots, a_n \rangle$  è una lista allora  $\forall i, j \mid 1 \leq i \leq j \leq n$  anche  $\langle a_i, \dots, a_j \rangle$  è una sottolista di  $l$ , ottenuta partendo dalla posizione  $i$  e prendendo tutti gli elementi fino alla posizione  $j$ . Inoltre, la lista vuota  $\langle \rangle$  è sottolista di qualsiasi lista.

### 4.2.1 Specifica sintattica

La specifica sintattica per il tipo di dato lista è la seguente.

- Tipi: lista, posizione, boolean, tipoelem
- Operatori:
  - `crealista` :  $() \rightarrow \text{lista}$
  - `listavuota` :  $(\text{lista}) \rightarrow \text{boolean}$
  - `leggilista` :  $(\text{posizione}, \text{lista}) \rightarrow \text{tipoelem}$
  - `scrivilista` :  $(\text{tipoelem}, \text{posizione}, \text{lista}) \rightarrow \text{lista}$
  - `primolista` :  $(\text{lista}) \rightarrow \text{posizione}$
  - `finelista` :  $(\text{posizione}, \text{lista}) \rightarrow \text{boolean}$
  - `succlista` :  $(\text{posizione}, \text{lista}) \rightarrow \text{posizione}$
  - `predlista` :  $(\text{posizione}, \text{lista}) \rightarrow \text{posizione}$
  - `inslista` :  $(\text{tipoelem}, \text{posizione}, \text{lista}) \rightarrow \text{lista}$
  - `canclista` :  $(\text{posizione}, \text{lista}) \rightarrow \text{lista}$

### 4.2.2 Specifica semantica

La specifica semantica per la struttura dati lista è la seguente.

- Tipi:
  - Lista: insieme delle sequenze  $l = \langle a_1, a_2, \dots, a_n \rangle$  con  $n \geq 0$ , di elementi di tipoelem dove l'elemento  $i$ -esimo ha valore  $a(i)$  e posizione  $\text{pos}(i)$ ;

– boolean: insieme dei valori di verità.

• Operatori:

- `crealista` =  $l'$ 
  - \* POST:  $l' = \langle \rangle$  lista vuota
- `listavuota`( $l$ ) =  $b$ 
  - \* POST:  $\begin{cases} b = true & l = \langle \rangle \\ b = false & l \neq \langle \rangle \end{cases}$
- `leggilista`( $p$ ,  $l$ ) =  $a$ 
  - \* PRE:  $p = pos(i) \quad 1 \leq i \leq n$
  - \* POST:  $a = a(i)$
- `scrivilista`( $a$ ,  $p$ ,  $l$ ) =  $l'$ 
  - \* PRE:  $p = pos(i) \quad 1 \leq i \leq n$
  - \* POST:  $l' = \langle a_1, a_2, \dots, a_{i-1}, a, a_{i+1}, \dots, a_n \rangle$
- `primolista`( $l$ ) =  $p$ 
  - \* POST:  $p = pos(1)$
- `finelista`( $p$ ,  $l$ ) =  $b$ 
  - \* PRE:  $p = pos(i) \quad 1 \leq i \leq n + 1$
  - \* POST:  $\begin{cases} b = true & p = pos(n + 1) \\ b = false & altrimenti \end{cases}$
- `succlista`( $p$ ,  $l$ ) =  $q$ 
  - \* PRE:  $p = pos(i) \quad 1 \leq i \leq n$
  - \* POST:  $q = pos(i + 1)$
- `predlista`( $p$ ,  $l$ ) =  $q$ 
  - \* PRE:  $p = pos(i) \quad 2 \leq i \leq n$
  - \* POST:  $q = pos(i - 1)$
- `inslista`( $a$ ,  $p$ ,  $l$ ) =  $l'$ 
  - \* PRE:  $p = pos(i) \quad 1 \leq i \leq n + 1$
  - \* POST:  $\begin{cases} l' = \langle a_1, \dots, a_{i-1}, a, a_{i+1}, \dots, a_n \rangle & 1 \leq i \leq n \\ l' = \langle a_1, a_2, \dots, a_n, a \rangle & i = n + 1 \\ l' = \langle a \rangle & i = 0 \text{ e } l = \langle \rangle \end{cases}$

- `canclista(p, l) = l'`
  - \* PRE:  $p = pos(i) \quad 1 \leq i \leq n$
  - \* POST:  $l' = \langle a_1, a_2, \dots, a_{i-1}, a_{i+1}, \dots, a_n \rangle$

Dalla specifica semantica emerge che per accedere a un elemento occorre conoscere la posizione. Questo nelle liste è possibile solo per il primo elemento, L'unico operatore che dà; per risultato la posizione è `primolista`. Per gli altri la posizione si ottiene conoscendo a priori la posizione dell'elemento precedente o seguente e applicando l'operatore `predlista` o `succlista`.

Dunque per accedere ad un generico elemento occorre scandire la lista a partire dal primo elemento.

L'operatore `listavuota` è ridondante visto che può essere sostituito da:

`finelista(primolista(l), l)`

Talvolta, l'operatore `primolista(l)` può avere implementazioni differenti, a volte ritorna l'elemento primo e non la posizione.

### Esempio : Eliminazione di duplicati

Questo è un esempio di come sia possibile, avendo a disposizione gli operatori definiti nell'algebra per il trattamento di una lista, risolvere un generico problema, senza sapere come la lista sia stata implementata.

Utilizziamo come linguaggio di programmazione il pascal e supponiamo di avere a disposizione degli operatori sul tipo `lista`.

Quello che vogliamo fare è eliminare gli elementi duplicati in una lista.

```

1 elimina_duplicati(lista l)
2   p = primolista(l)
3   while not finelista(p, l) do
4     q = succlista(p, l)
5     while not finelista(q, l) do
6       if leggilista(p, l) == leggilista(q, l) then
7         canclista(q, l)
8       q = succlista(q, l)
9     p = succlista(p, l)
10

```

Quindi abbiamo fatto riferimento alla struttura `lista` senza sapere i dettagli realizzativi.

### 4.2.3 Realizzazione

Per realizzare una lista distinguiamo due rappresentazioni fondamentali:

1. rappresentazione sequenziale;
2. rappresentazione collegata.

#### Rappresentazione sequenziale

Una lista può essere rappresentata usando un vettore e visto che il numero di elementi che compongono la lista può variare si utilizza una variabile *primo* per il valore dell'indice della componente del vettore in cui è memorizzato il primo elemento della lista.

Inoltre, si utilizza un'altra variabile *lunghezza* per indicare il numero di elementi di cui è composta la lista rappresentata.

Ad esempio data, una lista  $l$ :

$$l = \langle 4, 5, 1, 21, 45 \rangle$$

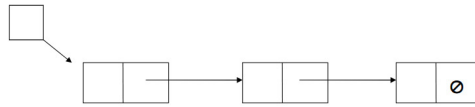
la variabile *primo* ha valore 1 e la variabile *lunghezza* ha valore 5.

Questa rappresentazione consente di realizzare molto semplicemente alcune delle operazioni definite per la lista ma crea problemi per l'inserzione e la rimozione di componenti, infatti se vogliamo aggiungere l'elemento 11 nella posizione 4,  $l = \langle 4, 5, 1, 11, 21, 45 \rangle$ , dovremmo spostare verso il basso il quarto e il quinto elemento.

#### Rappresentazione collegata

L'idea fondamentale della rappresentazione collegata di una lista è quella di memorizzare i suoi elementi associando ad ognuno di essi una particolare informazione, il *riferimento*, che permetta di individuare la locazione in cui è memorizzato l'elemento successivo. Per visualizzare tale rappresentazione si usa una notazione grafica in cui:

- gli elementi sono rappresentati mediante nodi;
- i riferimenti mediante archi che collegano nodi.



Si può notare che si usa un riferimento al primo elemento della lista, detto *riferimento iniziale* e un simbolo speciale  $\emptyset$  come riferimento associato all'ultimo nodo, nel caso in cui la lista sia vuota, tale simbolo compare direttamente al riferimento iniziale.